Thinking 8th

An original adaptation of Leo Brodie's Thinking Forth, for the 8th language

Editable manuscript master — work in progress

Contents

Preface

Thinking 8th is an attempt to do for 8th what Leo Brodie's *Thinking Forth* (1984) did for Forth: teach not the syntax of a language, but the way of thinking that makes it worth learning. Brodie's book is not a manual. It is an argument about how to design software — when to factor, what to name, how to hide the parts of a program most likely to change — illustrated with a language that makes those decisions unusually visible.

8th is a good language for the same kind of book, and a different one. It descends from Forth by way of Ron Aaron's Reva Forth, and it keeps the things that made Brodie's argument possible: words instead of functions, a data stack instead of named parameters, an interactive interpreter instead of a batch compiler. A reader who has never seen Forth will still recognize the shape of the ideas here. But 8th is not a 1980s Forth wearing new syntax. It has real namespaces instead of a naming convention. Its "variables" are one of several built-in container types, reference-counted and garbage collected, not raw addresses into memory you manage yourself. It runs the same source file on a desktop, a phone, and a server. Where those differences change the lesson, this book says so, instead of pretending 8th is Forth in disguise.

This is **not** a mechanical translation of *Thinking Forth*. Each chapter starts from what Brodie was actually trying to teach, separates that lesson from the Forth-specific mechanics he used to teach it, and then asks how the same lesson is naturally expressed in idiomatic 8th. Sometimes the answer is "almost exactly the same code, different words." Sometimes it is "8th already has a real feature for the thing Forth programmers had to improvise." Occasionally the honest answer is "this particular Forth technique doesn't have a natural 8th analogue, and here's why." All three answers show up in this book.

Thinking Forth is available at the Thinking Forth SourceForge project under a Creative Commons Attribution-NonCommercial-ShareAlike 2.0 license, and a copy of its original LaTeX source is kept in this repository, under `thinking-forth-1.0/`, as reference material. This book is an original adaptation: original prose and original, independently verified 8th code, inspired by Brodie's structure and spirit rather than copied from his text. It carries the same non-commercial, share-alike spirit forward.

Every code example in this book that can be run has been run, against the 8th distribution it was written against, and its actual output is shown alongside it. Where an example can't reasonably be executed — because it needs a GUI, a network, hardware, or some other environment this book doesn't assume — that is said plainly, rather than guessed at.

Read the following section, "A Note on Notation," next; it explains a handful of things about how 8th source code is written in this book that will save you from misreading the very first examples.

A Note on Notation

If you have ever read about Forth, skip this page at your peril. 8th looks enough like Forth that old habits will actively mislead you in a few specific spots. This page exists so the very first code example in Chapter 1 doesn't confuse you.

Comments: \, not `( ... )`

In most Forths, `( n1 n2 -- n3 )` is a comment — a stack-effect diagram that the compiler skips over. In 8th, **parentheses are not comments**. `( ... )` compiles the words inside it into an anonymous, callable block of code and leaves a reference to that block on the stack. This is a real, load-bearing feature (it is how 8th builds things like loop bodies and callbacks), and it means that a stray `( n1 n2 -- n3 )` in the middle of an 8th word will not be silently ignored — it will try to compile `n1`, `n2`, `--`, and `n3` as words, and fail.

8th's comment character is the backslash, `\`, which runs to the end of the line (`--` does the same thing, in the SQL-comment style, and both can be used interchangeably). There is also a multi-line comment, `(* ... *)`, which nests. This book uses `\` throughout, following the convention used in 8th's own tutorials and sample code:

```
\ this whole line is a comment
: greet   \ -- ; prints a greeting
  "hello" . cr ;
```

Stack-effect comments (Brodie's `( n1 n2 -- n3 )`, called an **SED**, "stack-effect diagram," in 8th's own documentation) are written after a backslash, in the same relative position Brodie uses them: right after the word's name, before its body.

Words, not functions

Brodie calls the unit of a Forth program a "word," and so does 8th's own manual: "the same as a function, procedure, or routine in other languages." This book uses "word" throughout, for the same reason Brodie did — because in both languages, a defining name and an executable unit are the same thing, and there is no separate machinery of "calling" it.

Namespaces: `n:`, `s:`, `a:`, and friends

8th ships with a large vocabulary of built-in words, grouped into **namespaces** — `n:` for numbers, `s:` for strings, `a:` for arrays, `m:` for maps, and so on. `n:+` is addition on numbers; `s:+` is string concatenation. You will see this prefix convention constantly and it is not optional decoration — `+` by itself is not a word in 8th. Chapter 1 says more about why this matters.

Variables: `var,` `@`, `!`

A variable in 8th is declared with `var` (initialized to `0`) or `var,` (initialized to whatever is on top of the stack), and its contents are read with `@` and written with `!`, exactly as in Forth:

```
0 var, count
5 count !
count @ .          \ prints 5
```

The one habit to unlearn: the name `count` does **not** stand for the value stored in the variable. It stands for a reference to the variable itself. `count @` fetches the value; bare `count` does not. Forth programmers already know this distinction for `VARIABLE`, so this is one of the places where prior Forth experience transfers directly.

Booleans

8th has real `true` and `false` words, not "any nonzero value." This book uses them rather than raw `-1/0` or `1/0`, which is also the idiomatic style in 8th's own sample code.

With that out of the way — on to the philosophy.

Chapter 1: The Philosophy of 8th

8th is a language, a runtime, and — like the language that inspired it — an implied argument about how software ought to be built. Nobody wrote that argument down as a manifesto. It shows up instead in the shape of the language itself: what's easy to say, what's hard to say, and what the language simply refuses to let you say. Before looking at 8th code, it's worth asking where that shape came from, and what problem it was built to solve.

A Short History of Trying to Make Software Manageable

Early programs were bit patterns entered by hand — correct-or-not was the only question anyone asked. As machines and budgets grew, a second question appeared alongside correctness: could the program be *changed* without breaking it? Decades of language design have really been one long answer to that second question.

Assemblers gave instructions names instead of bit patterns. Macro assemblers gave repeated sequences of instructions names too. High-level languages broke the one-to-one link between what you typed and what the machine did, so `X = Y * (456/A) - 2` could stand for a dozen machine instructions at once. Structured programming broke large problems into modules with one entrance and one exit, so a reader could reason about a piece of a program without holding the whole thing in their head.

Each of these was a real advance, and each one eventually ran into the same wall: a program decomposed by what it *does* — read the record, edit the record, write the record — falls apart the moment something about *how* it does it changes. Change the record's layout and you're back in all three modules at once, because all three modules knew that layout.

In 1972 David Parnas proposed a different criterion for drawing module boundaries: not sequence, not control flow, but **what is likely to change**. A module's job, in this view, is to hide one such likely-to-change thing — a data layout, an algorithm, a piece of hardware — behind a small set of routines that the rest of the program uses instead of touching the thing directly. Get the boundary right, and a change stays inside the module that owns it. Barbara Liskov and Stephen Zilles gave the same idea a name a few years later: *data abstraction*. Their example was a stack: routines to push, pop, and test for empty, with the actual representation hidden behind them.

This is the idea 8th — like Forth before it — makes unusually natural to follow: not a design pattern you have to remember to apply, but close to the default shape of an 8th program. Build out of small, named words with data passed implicitly between them, and you are already most of the way toward decomposing by what might change, whether you set out to or not.

Words Are the Unit, Not Functions or Modules

Here is a complete, if small, 8th program:

```
true var, hurried

: hurried?  hurried @ ;
: cereal    "cereal\n" . ;
: eggs      "eggs and bacon\n" . ;
: clean     "wash up\n" . ;

: breakfast
  hurried? if cereal else eggs then clean ;

breakfast
```

Running it (see [`code/ch01/breakfast.8th`]) prints:

> *cereal*
> *wash up*

`breakfast` isn't a subroutine call away from `hurried?`, `cereal`, `eggs`, and `clean` — it's *built out of* them, the same way a sentence is built out of words rather than referring to them from a distance. 8th's own manual defines its basic unit of execution this way: "the same as a function, procedure, or routine in other languages" — but that undersells the difference in practice. There is no separate mechanism for defining a subroutine, declaring a variable, or writing the main program. A variable declaration (`true var, hurried`), a word that reads it (`hurried?`), and the word that ties them together (`breakfast`) are all just words, invoked the same way, by name. There is no `main()` that is structurally different from the functions it calls.

Two things about 8th make this possible, and Leo Brodie identified both of them in Forth forty years ago: calls are implicit, and so is data passing.

Calls are implicit. You don't write `call cereal`; you write `cereal`. Every name 8th finds — a word, a variable, a constant — carries its own instructions for what to do when invoked. There is exactly one way to invoke anything: say its name.

Data passing is implicit. `hurried?` doesn't take `hurried` as an argument in the conventional sense; it fetches the value and leaves it where the next word, `if`, expects to find it — on top of the data stack. `breakfast` never mentions the stack at all. It reads as a flat sequence of decisions: *are we hurried? if so, cereal, otherwise eggs; either way, clean up afterward.* The stack is the plumbing that makes this reads-like- a-sentence quality possible, and once you trust it, you stop thinking about it, the same way you stop thinking about which register a value sits in when you write `a + b` in almost any other language.

Because arguments travel on a shared stack instead of being named and declared, any word can be built out of any other words without either one knowing about the other's internals. That's what lets `breakfast` read as a single, flat idea instead of a nested tree of calls — and it's also *exactly* Parnas's information-hiding, arrived at as a side effect of how the language passes data around, rather than as a discipline layered on top.

Namespaces: A Lexicon You Don't Have to Invent

Brodie's book coins a term for a set of words that together hide one component's details from the rest of an application: a **lexicon** — "your interface with the component from the outside." In classic

Forth this is purely a design convention; nothing in the language enforces it or even knows it exists. A word belonging to a "stack" lexicon and a word belonging to a "queue" lexicon live in exactly the same flat dictionary, distinguished only by whatever naming discipline the programmer imposes.

8th takes the same idea and builds it into the language as a real feature: the **namespace**. Its own manual defines a namespace as "a vocabulary of (usually) related words" — which is Brodie's definition of "lexicon," independently arrived at, formalized, and enforced by the interpreter rather than left to convention. Every built-in word that operates on numbers lives in the `n:` namespace (`n:+`, `n:-`, `n:1-`, `n:+!` — you used two of these already, in the variable example above). Strings live in `s:`, arrays in `a:`, maps in `m:`, and so on. When you write your own component, you can give its words their own namespace prefix the same way, and 8th's `with: / ;with` lets you temporarily bring a namespace's words into scope unprefixed, for readability, without ever losing the boundary between components.

This is the one place in this chapter where "the natural 8th approach" is genuinely, structurally different from Forth's, rather than a change of spelling: what Brodie has to argue readers *into* doing — decompose your program into small lexicons with clean boundaries — 8th's own standard library already does, pervasively, as ordinary practice you'd have to work to avoid.

Hiding the Construction of a Data Structure

Brodie's central example of information-hiding is a variable called `APPLES`, used to tally apples, that later needs to become two variables — one for red apples, one for green — without changing a single line of code that already uses `APPLES`. The trick works in 8th exactly the way it works in Forth, for the same underlying reason: a variable's *name* and the *value it produces when read* are two different things, so the second can be redefined without disturbing the first.

Start with a plain variable:

```
0 var, apples

20 apples !
apples @ . cr                 \ => 20

1 apples n:+!
apples @ . cr                 \ => 21
```

Now suppose, after code elsewhere already depends on `apples`, you discover you need two tallies — red and green — selected by which color is "current":

```
0 var, color                  \ which color is "current"?
0 var, reds
0 var, greens

: red    reds color ! ;
: green  greens color ! ;

: apples  color @ ;
```

`apples` is no longer a variable at all — it's a *word* that returns whichever variable is currently selected. But because `apples @`, `apples !`, and `apples n:+!` all still work exactly as before, nothing that used `apples` needs to change:

```
red
20 apples !
apples @ . cr                   \ => 20

green
5 apples !
apples @ . cr                   \ => 5

1 apples n:+!
apples @ . cr                   \ => 6

red
apples @ . cr                   \ => 20   (still there, untouched)
```

This is `code/ch01/apples.8th`, run and verified against the actual output shown above. One detail worth knowing about, because it's genuinely useful rather than merely cosmetic: when the file redefines `apples` from a variable into a word, 8th prints a warning to the console —

> *Redefining: user:apples*

— and then proceeds. This is 8th telling you, out loud, exactly the thing this example is about: you have replaced an existing name with a new meaning. In a large program, that warning is a safety net; here, it's confirmation that the trick worked.

What made this possible is the same pair of features from the last section, applied to *nouns* instead of *verbs*: `apples` is a word, so calling it is implicit; and it produces a reference on the stack rather than a name in the source text, so `@`, `!`, and `n:+!` don't need to know or care whether that reference came from a plain variable or from three lines of logic. 8th, like Forth, doesn't force you to distinguish between "a thing" and "an action that produces a thing" — a word can play either role, and nothing about how you invoke it gives away which one it's playing today.

Is 8th a High-Level Language?

By the standard measure — does it hide the correspondence between source code and machine operations — 8th is unambiguously high-level; it runs on top of 8th's own interpreter engine, not on a specific processor's instruction set, and the same source file runs unmodified on a desktop, a phone, or a server. By another standard measure — strict syntax checking that catches your mistakes before you run the program — 8th, again like Forth, does almost none. Write `apples red` where you meant `red apples` and 8th won't stop you; it will simply do what you said, which is not what you meant.

The trade this makes is the same one Brodie described in 1984: in exchange for very little static protection, you get a language with no fixed grammar to fight. Adding a new *kind* of word — a new control-flow construct, a new way of defining something — is not a special, harder kind of programming in 8th; it's what the words `if`, `var`, and `:` themselves are, examples of an extension mechanism available to you as much as it was available to whoever wrote those particular words. There is no wall between "the language" and "code you wrote."

8th also leans further into interactivity than most languages that came after Forth. There's no edit-compile-link-test cycle; you type a phrase and the interpreter answers immediately, whether that phrase is a whole program or three words you're checking the behavior of. This turns "design" and "test" into the same activity rather than sequential phases — you can write the outermost, most abstract word of an application first and give its supporting words trivial, placeholder definitions, running the whole thing end-to-end from day one, then replace the placeholders one at a time as the real implementation gets built underneath them. This isn't a workaround for the absence of a proper design phase; it *is* a design method, and it depends on exactly the same word-at-a-time, implicit-calls, implicit-data-passing properties this chapter has been describing.

Performance and Portability

Brodie spent several pages of the original chapter arguing that Forth, despite its unusual appearance, was competitive with assembly language in size and speed, thanks to a compilation technique called threaded code. That argument doesn't carry over to 8th unchanged, and it would be dishonest to pretend otherwise: by 8th's own documentation, compiling a word packs what it needs into an internal code cache rather than emitting native machine instructions (an earlier version of 8th could generate native code, but that path was dropped because of restrictions on the iOS platform) — and, by explicit design choice, 8th performs no optimization on your code at all, apart from tail-call elimination. The reasoning given is not "we haven't gotten to it yet"; it's that an optimizer can silently change a program's behavior, and that the most effective optimization available is still a programmer choosing a better algorithm.

What 8th trades raw execution speed for is something Forth in 1984 could not offer: the same source file, unmodified, targets desktop operating systems, mobile platforms, and embedded devices from one implementation. Where Brodie's Forth achieved portability across hardware by being small enough to reimplement on each new target, 8th achieves it by being one implementation that already runs everywhere. Different eras, different scarce resource, same underlying value — write the logic once, in a language that gets out of the way.

Summary

Strip away the unfamiliar punctuation, and 8th is making the same wager Forth made: that a program built entirely out of small, named, freely composable words — with data flowing between them implicitly, on a stack, rather than declared and passed by hand — makes Parnas's kind of change-driven decomposition unusually natural to fall into, rather than something you have to impose on top of subroutines, modules, or objects with explicit interfaces. Where Forth left the discipline of grouping those words into coherent components ("lexicons," in Brodie's term) up to the programmer, 8th builds a formal version of the same idea into the language as namespaces. Where Forth achieved portability by being small and easy to port, 8th achieves it by being one implementation that already runs everywhere you're likely to want to deploy.

None of this is free. You give up static type-checking, a fixed grammar to lean on, and — compared to hand-tuned native code — raw speed. What you get in exchange is a language with almost nothing between your intent and the running program: no ceremony for declaring a subroutine, no boilerplate for passing arguments, no separate compile step standing between a change and seeing it run. The rest of this book is about what that trade lets you build, and how to build it well.

Chapter 2: Analysis

Nobody agrees on how many phases software development has. Brodie counted nine, from discovering requirements down to maintenance, and then spent the rest of his second chapter demonstrating that Forth programmers routinely ignore the ordering. The same is true of 8th, for the same underlying reason: a language where you can write one word, test it in the running system, and move on doesn't force analysis, design, and implementation into separate, sequential phases. It lets them interleave.

That's a genuine advantage, not an excuse to skip thinking. This chapter is about what to think about before — and while — you write 8th code that matters.

Iteration Beats Prediction

Brodie interviewed working Forth programmers throughout the 1980s, and a pattern emerged that had nothing to do with Forth specifically: the programmers who did the best work were not the ones who planned the most, or the ones who planned the least, but the ones who treated their early code as a question rather than an answer. Build the smallest thing that tells you something true about the problem, show it to the people who'll actually use it, and let what you learn reshape the next version. Planning still matters — nobody plans nothing on a project worth doing — but it has diminishing, and eventually negative, returns. Past a certain point, planning is a way of avoiding the discovery that your plan was wrong.

8th's whole design leans into this. There's no build step standing between "I changed the code" and "I can see what it does now." A word you're unsure about can be defined with a placeholder body — print its own name, return a fixed value, do nothing — and wired into the rest of the program immediately, so that the shape of the whole system is testable from the very first day, long before every part of it is real. Later chapters return to this technique in more depth; this chapter is about how to decide *what* those words should be before you write any of them.

What Analysis Actually Produces

Whatever you call the phase, analysis has to answer three questions before serious implementation starts:

1. What does the system actually need to do, and what constraints (time, memory, an existing device it has to talk to, a deadline) bound the answer?
2. What's the simplest model of a solution that satisfies those needs?
3. Roughly, what will it cost — in time, and in the resources the target system actually has — to build that model?

The rest of this chapter is about the second question, because it's the one a language can actually help with.

Sketching Interfaces in Words, Not Diagrams

A common technique for the first question is the data-flow diagram: circles for operations, arrows for the data moving between them. It's a useful tool for explaining a design to someone who doesn't read code. But if the person you're explaining it to *does* read code, a word-based language can often skip the diagram and go straight to something almost as readable, and far more useful, because you can run it:

```
false var, garage-full?

: space-available?  garage-full? @ not ;
: let-in    "Welcome -- take a ticket.\n" . ;
: turn-away "Sorry, we're full.\n" . ;

: admit-car  \ --
  space-available? if let-in else turn-away then ;
```

This is a complete, if trivial, working sketch of a parking garage's entry policy — not pseudocode dressed up to look like 8th, but real, runnable 8th in which the interesting decision (`space-available?`) is separated from the actions it chooses between (`let-in`, `turn-away`), and the top-level word (`admit-car`) reads as a flat sentence describing the policy. Nothing here commits you to how a car's arrival is actually detected, how a ticket is actually printed, or how `garage-full?` actually gets set — those are implementation details you can fill in underneath this sketch, one word at a time, without changing `admit-car` at all. Run it and it behaves exactly as the words suggest:

```
admit-car
true garage-full? !
admit-car
```

prints

```
Welcome -- take a ticket.
Sorry, we're full.
```

That's the whole point. A design sketch you can execute catches a misunderstanding immediately — try the phrase, watch what happens — instead of after the diagram has been approved and handed off for implementation.

Defining the Rules: From Prose to a Decision Table

Interfaces are usually the bulk of an analysis. Occasionally, though, an application has genuine *rules* — logic complicated enough that a sentence of English doesn't capture it safely. Suppose our garage's exit fee depends on when you parked (weekday daytime, weekday evening, or weekend), how many hours you stayed, and whether you used valet service. Written as a paragraph, the rate schedule reads about as well as a tax form: "During weekday daytime hours the charge is $4.00 for the first hour and $2.00 for each additional hour... on weekday evenings, $2.00 for the first hour and $1.00 for each additional hour... on weekends, a flat $1.00 per hour... valet service adds a flat $5.00 regardless of the hour or day." It's not wrong, it's just hard to check for gaps or contradictions by eye.

Turning the same rule into nested conditionals doesn't help much — you'd get a wall of `if/else` three or four levels deep, repeating the "additional hour" logic inside every branch of the "which tier" decision, which obscures the one fact that actually matters for simplifying the problem: **the per-hour rates depend only on which tier you're in, and nothing else.** A **decision table** — tier along one axis, first-hour and additional-hour rates along the other — makes that fact visible at a glance, in a way prose and nested conditionals both bury.

	first hour	**additional hour**
weekday day	$4.00	$2.00
weekday evening	$2.00	$1.00
weekend	$1.00	$1.00

Once the rule is a table, 8th lets you implement it as an actual table, looked up by index, rather than as a chain of comparisons pretending to be one. 8th's `caseof` takes a container and an index (or a string key, for a map) and returns whatever is stored there — a number, a string, or, if it's a word, the *result of calling it*. Read as "look this up," a decision table and a `caseof` array are the same idea:

```
0 constant DAY
1 constant EVENING
2 constant WEEKEND

0 var, tier
false var, valet?

: set-day      DAY tier ! ;
: set-evening  EVENING tier ! ;
: set-weekend  WEEKEND tier ! ;
: valet-on     true valet? ! ;

[ 400 , 200 , 100 ] constant first-hour-rates   \ cents
[ 200 , 100 , 100 ] constant addl-hour-rates    \ cents

: first-hour-rate   \ -- cents
  first-hour-rates tier @ caseof ;

: addl-hour-rate    \ -- cents
  addl-hour-rates tier @ caseof ;
```

The valet surcharge isn't part of the table at all — it doesn't depend on the tier or the hour, so tying it to either would be exactly the kind of false coupling Chapter 1 warned about. It's its own small word:

```
: valet-surcharge   \ cents -- cents
  valet? @ if 500 n:+ then ;
```

And the whole fee is a composition of these three pieces, matching the shape of the table instead of hiding it:

```
: parking-fee  \ hours -- cents
  1 n:-  addl-hour-rate n:*
  first-hour-rate n:+
  valet-surcharge ;
```

This is [`code/ch02/parking-fee.8th`](), executed and checked against hand-calculated expectations:

```
set-day       3 parking-fee . cr    \ => 800   (400 + 2*200)
set-evening   3 parking-fee . cr    \ => 400   (200 + 2*100)
set-weekend   5 parking-fee . cr    \ => 500   (100 + 4*100)
valet-on
set-day       1 parking-fee . cr    \ => 900   (400 + 0 + 500)
```

Notice what happened to the + in the table: in the paragraph-of-prose version it appeared nine times, once per cell. In the factored version it appears exactly twice — once combining the two rate components, once adding the surcharge — because the table stopped being nine separate facts and became one small idea (a per-tier rate) applied uniformly. That collapse from nine cases to one idea *is* the analysis. The 8th code is just where the analysis stops being deniable: if the factoring is wrong, `parking-fee` gives you the wrong number, immediately, rather than a diagram nobody double-checked.

Data Structures and the Limits of Generality

Sometimes analysis has a third job: deciding what to remember, not just what to do. A parking garage that only ever admits and charges one car at a time doesn't need much of a data structure. One that needs to know which of its two hundred spaces are occupied, by which ticket number, since what time, is a different problem — and *that* decision (one record per space? one growing log of entries and exits?) belongs in analysis, before any code commits you to it, because it's exactly the kind of thing Chapter 1 called "likely to change": today it's spaces in a garage, next year it might be spaces in three garages, or hourly and monthly permit-holders sharing the same lot.

The temptation, faced with that uncertainty, is to generalize: build a data structure that could handle any garage configuration anyone might ever want. Resist it. A solution sized for problems you don't have yet is usually harder to understand, harder to verify, and — because generalization multiplies the number of cases that interact with each other — no easier to change than one sized for the problem you actually have, built so that the parts likely to change are hidden behind a small lexicon of words, the way `tier`, `first-hour-rate`, and `valet?` hide the rate structure above. Simple and changeable beats general, almost every time.

Summary

Analysis, in 8th as in Forth, isn't a document you produce before coding starts — it's the activity of finding the smallest accurate model of the problem, however long that takes, and 8th's short feedback loop makes it cheap to test that model as you refine it rather than only after it's finished. Interfaces are usually best expressed directly as words with placeholder bodies, executable from day one. Genuinely complicated rules are best captured as decision tables, and 8th's `caseof` lets a decision table survive into the running program as an actual table instead of dissolving into a maze of conditionals. And whatever data structures the problem needs should be sized to the problem you have, not the one you can imagine — because the parts you can't predict are exactly the parts you'll want to have hidden behind a word, ready to change.

Chapter 3: Preliminary Design and Decomposition

Analysis tells you what a program has to do. Preliminary design is the next step: deciding what pieces it should be built from. Get this step right and implementation is a series of small, well-defined problems. Get it wrong, and you spend implementation discovering the right decomposition anyway — the hard way, by fighting code that doesn't want to bend the way the requirements just bent.

Brodie describes two ways to divide a program into pieces. This chapter works through both, with one running example, in 8th.

Two Ways to Cut Up a Problem

The first way is **decomposition by component**: group words by the thing they know about — a data structure, a device, a rule — regardless of when in the program's execution that knowledge gets used. This is the approach Chapter 1 already argued for: components as Parnas-style likely-to-change boundaries, given a name (a namespace, in 8th) and a small set of words as their public face.

The second is **decomposition by sequential complexity**: since a word has to exist before anything can call it, a program built word by word tends to arrange itself from simplest to most capable, the way a textbook moves from basic facts to advanced ones. This isn't really a design choice — it's a consequence of writing in a word-based language at all — but it has implications worth understanding, including one genuine wrinkle: sometimes a foundational word needs to call something that, by the "simplest first" ordering, hasn't been written yet.

Both approaches show up in the example below.

Decomposition by Component: A Thermostat

Suppose the problem is a thermostat: read a temperature, decide whether to heat, cool, or do nothing, and let a person override the decision manually. Before writing any of that, ask what a *component* boundary looks like here. One candidate jumps out immediately: whatever "the current mode" is and how it gets changed is exactly the kind of thing likely to grow more rules later (a minimum-run timer, a "don't switch modes twice in five minutes" guard) — so it belongs behind its own small set of words, not scattered through whatever code happens to decide when to heat or cool.

```
0 constant IDLE
1 constant HEATING
2 constant COOLING

IDLE var, mode

: mode@      \ -- mode
  mode @ ;

: set-mode   \ new-mode --
  dup mode@ n:= not if
    dup mode !
  else
    drop
  then ;
```

`set-mode` isn't just "store a number in a variable" dressed up. Look at what it's already doing: it's the one place in the whole program that ever sees *both* the old mode and the new one at the same moment, because it's the one place any code goes through to change the mode at all. Nothing outside this handful of words ever touches the `mode` variable directly — not even to read it, which is what `mode@` is for. That discipline is worth naming, because it's what Chapter 1 called an interface component: whatever data two or more other parts of the program need to share should live behind its own words, not be reached into directly, precisely so that a rule about *how* it's shared (like "only change it if it's actually different") has exactly one place to live.

Now the two things that actually want to change the mode. The automatic decision:

```
: decide-mode  \ degrees -- new-mode
  dup 68 n:< if
    drop HEATING
  else
    76 n:> if COOLING else IDLE then
  then ;
```

And a person overriding it by hand:

```
: heat        HEATING set-mode ;
: cool        COOLING set-mode ;
: hvac-idle   IDLE    set-mode ;
```

Notice that `heat`, `cool`, and `hvac-idle` don't touch `mode` at all — each is just a name for "call `set-mode` with a particular constant." That wasn't planned in advance; it fell out once `set-mode` existed as its own word, because writing "what changes the mode" three different ways (once automatically, three times manually) would have meant deciding, three separate times, whether the change was worth acting on. One shared word underneath both the automatic and the manual path means that decision gets made once. This is the same move Brodie's book makes with an editor whose `INSERT` command turns out to be nothing more than "make room, then overwrite" — a word that already existed, doing a job nobody had thought to name yet. It's not something you plan for up front so much as something you notice once you've written the pieces down and looked at what they have in common.

A Change in Plan

Here's where a component-based design earns its keep. Suppose the requirement changes: instead of announcing the mode every cycle, the thermostat should only speak up when the mode actually *changes* — nobody wants a log line every ten seconds saying "still heating."

Because `set-mode` already sees both the old mode and the new one, it already *has* the information this change needs. Adding it costs one line:

```
[ "idle" , "heating" , "cooling" ] constant mode-names

: mode-name  \ mode -- s
  mode-names swap caseof ;

: set-mode   \ new-mode --
  dup mode@ n:= not if
    dup mode !
    mode-name "now %s\n" s:strfmt .    \ <-- the new line
  else
    drop
  then ;
```

Nothing else in the program changes. `heat`, `cool`, `hvac-idle`, and `decide-mode` are untouched, and every one of them picks up the new, quieter logging behavior automatically, because every one of them was already going through `set-mode`.

Compare that to what would have happened if the mode had been changed directly wherever it was decided — three lines in the manual-override words, one more inside whatever called `decide-mode` — with a "print the mode" step tacked on after each one, the way a flowchart-driven design naturally accumulates: one box for "decide," a separate box for "report," wired together by the arrows between them. Adding "only report when it changed" to *that* design means the "did it change" check either gets duplicated at every call site, or the previous mode has to be threaded through the control flow as extra state so the reporting step can compare against it — extra plumbing whose only job is to reconnect two things that a shared word would have kept connected for free. The component version didn't dodge that problem by being cleverer. It dodged it by already having exactly one place where the answer to "did the mode change?" was knowable without asking around.

[`code/ch03/thermostat.8th`] exercises this with a run of temperature readings. Reading the code rather than running it first: `60`, then `61`, then `72` should produce two mode changes and one silent repeat —

- `60` → HEATING (a change: `set-mode` logs `now heating`)
- `61` → HEATING again (no change: silent)
- `72` → IDLE (a change: `set-mode` logs `now idle`)

which is exactly what the actual run confirms further below, alongside a fourth reading this chapter isn't ready to explain yet.

Decomposition by Sequential Complexity, and Its One Wrinkle

The thermostat's words, read top to bottom, go from simplest to most capable: constants, then the mode variable, then `set-mode`, then the words built on top of it. That ordering isn't a style choice — 8th, like Forth, requires a word to be defined before anything can refer to it, so a program built up word by word naturally reads like a textbook, elementary material first.

Occasionally that ordering fights you. Suppose the sensor component needs to flag an implausible reading — a wildly out-of-range number that suggests a wiring fault — but the code that actually knows how to *handle* that situation (log it, alert someone, whatever a future diagnostics component decides to do) doesn't exist yet, and arguably shouldn't be designed until there's a real diagnostics story to design it around. The foundational sensor code is written first; the advanced response to it comes later. But the sensor code still needs to call *something* when it sees a bad reading, today, before that something has been written.

8th's answer to this is `defer:` — a word declared now, whose body is supplied later:

```
defer: on-bad-reading   \ degrees --

: plausible?  \ degrees -- flag
  dup -20 n:> swap 120 n:< and ;

0 var, sensor-temp

: read-temp  \ -- degrees
  sensor-temp @
  dup plausible? not if dup on-bad-reading then ;
```

Until something is attached to it, `on-bad-reading` is silently a no-op — `read-temp` compiles and runs correctly with no diagnostics component in sight. Later, once that component is designed, it attaches itself with `w:is`:

```
: report-bad-reading  \ degrees --
  "sensor reading %d looks implausible -- check wiring\n" s:strfmt . ;

' report-bad-reading w:is on-bad-reading
```

From this point on, every call to `read-temp` that sees an implausible value invokes `report-bad-reading` — without `read-temp` having been touched, and without the sensor component needing to know, when it was written, what "diagnosing a bad reading" would eventually mean. Running the same reading (999, well outside a plausible range) before and after this assignment shows the difference directly:

```
\ before the diagnostics component is wired in:
999 sensor-temp ! auto-cycle
\ => "now cooling"   (mode genuinely changes; no diagnostic — the hook
\    is still a no-op)

\ after ' report-bad-reading w:is on-bad-reading :
999 sensor-temp ! auto-cycle
\ => "sensor reading 999 looks implausible -- check wiring"
\    (mode is already "cooling", so no "now ..." log this time --
\    only the diagnostic fires)
```

This is [`code/ch03/thermostat.8th`] in full; running it start to finish prints exactly:

```
now heating
now idle
now cooling
sensor reading 999 looks implausible -- check wiring
final mode: cooling
```

`defer:` is a narrow tool for a narrow problem — a genuine forward reference, not a general substitute for planning ahead — but it means the order you'd naturally *write* a program (foundations first) doesn't have to match the order in which every dependency becomes known.

The Limits of "Level" Thinking

It's tempting, once a program is split into "foundational" and "advanced" pieces, to treat that split as a hierarchy you must climb in order — design the bottom first, then the middle, then the top. Nothing about component decomposition requires that. The thermostat example above was written `mode`/`set-mode` first only because that made the clearest starting point for this chapter — but `decide-mode` could just as easily have been written and tested first, standing on nothing but plain numbers on the stack, long before `read-temp` or `sensor-temp` existed:

```
60 decide-mode .    \ works today, no sensor required
```

Pick whichever piece gives you the most useful feedback fastest — the part you're least sure about, the part a stakeholder most needs to see working, the part whose difficulty will tell you whether the rest of the project is easy or hard. "Foundational" and "advanced" describe where a word ends up in the finished dependency graph, not the order you're obligated to design them in.

A related trap is building an interface too narrow to reach the thing behind it. If a component exposes only the three or four operations its first caller happened to need, and hides everything else — including information a *later* caller turns out to need but the component's author never anticipated — the later caller is stuck. It can't extend the component's own tools, because it doesn't have access to them; it can only work around the outside of a boundary that was drawn too tight. The fix isn't to expose everything (that defeats the point of having a boundary at all) — it's to expose the tools the component is built from, not just the one function that happened to be needed first, so a future caller with a slightly different need can compose those tools instead of reimplementing them from scratch outside the boundary. `mode@` above is a small instance of this: it costs one line to expose, and it means a future component that only needs to *read* the mode — a display, a log, a scheduler — never has reason to reach past `set-mode` and touch the variable directly.

Brodie's own version of this chapter, writing in 1984, extends the point into a broader argument against "objects" — meaning, in his usage, a single word that takes a selector parameter and internally dispatches to one of several behaviors, the way an old COM or CORBA object might. His concern was narrow and specific even though the term he used for it is broad: such a word has to contain its own internal decision structure to figure out which behavior you meant, and it can't be extended by adding a new named word the way a namespace of many small words can — you have to modify the dispatcher itself. That's a real, fair comparison between *that specific shape* (one name, an internal switch) and 8th's usual shape (many names, no switch needed because the name already picked the behavior). It is not a fair comparison between 8th and object-oriented programming in general — most OOP languages built after 1984 don't work by internal selector-dispatch either, and later editions of Brodie's own book say as much. The lesson worth keeping is narrower than "objects are bad": a word that has to ask "which of my several jobs am I doing this time?" is doing work that a well-factored set of separately named words doesn't have to do — regardless of what you call the wider style it's part of.

Summary

Preliminary design means deciding what pieces a program needs before writing them. 8th supports two ways of arriving at those pieces: grouping by what a piece knows (decomposition by component, which is where the real design work happens) and the ordering a word-based language imposes anyway (decomposition by sequential complexity, which `defer:` lets you escape when a foundational piece genuinely needs to call something that isn't written yet). The thermostat example showed both at once: a shared `mode` component discovered by noticing that several separate callers wanted the same underlying action, an interface disciplined enough (`mode@`, `set-mode`) that a real requirement change cost one line instead of a redesign, and a forward-referenced hook that let a foundational word call forward into a component that didn't exist yet. None of this required planning the whole dependency graph before writing any code — only writing each piece where its boundary was actually the right one to draw.